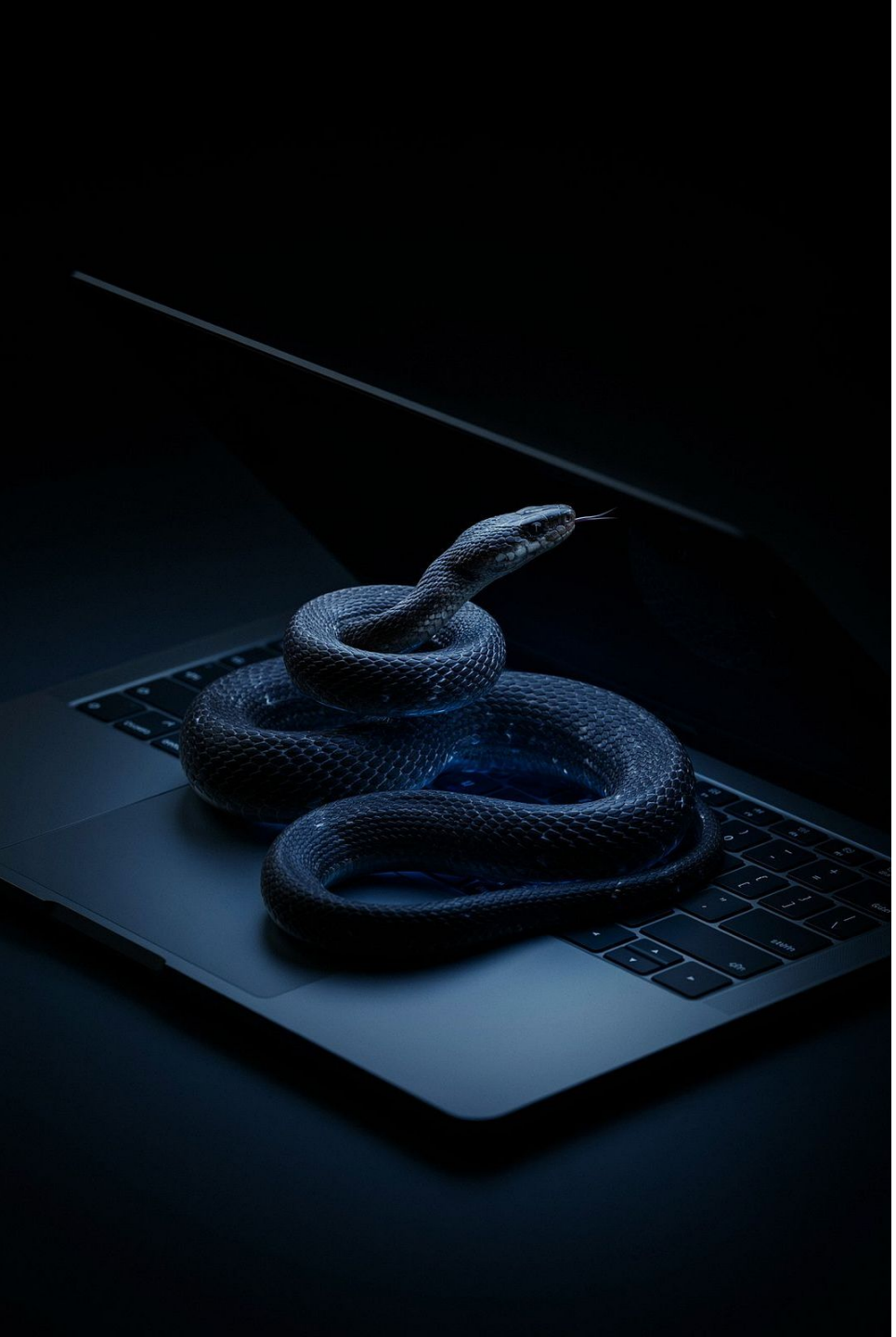


SPRINT 4 · PYTHON PARA IA · S04.1

# Python: entorno y lenguaje

TurisData calcula sus cifras a mano en una hoja de cálculo. Esta semana lo convertimos en un programa que cualquiera puede ejecutar y comprobar.

# Qué vamos a construir (el reto)

**Reto «Herramientas internas»:** un script kpis.py que lee el CSV de reservas, calcula 3 KPIs, avisa si el fichero está mal y guarda un JSON.

### Qué usaremos

Solo Python estándar: variables, funciones, listas, diccionarios, csv, json, excepciones.

### Cómo sabremos que va bien

python test_kpis.py en verde: cada prueba es un assert.

### Cómo lo entregaremos

Carpeta con kpis.py, test_kpis.py, README.md y varios commits en Git.

# Dónde se ejecuta Python

| Dónde | Cuándo | Ojo con |
|---|---|---|
| Google Colab | En el aula: no hay que instalar nada | Al cerrar la sesión se pierde el estado |
| Jupyter / VS Code | En tu equipo, con tus propios ficheros | Hay que instalar Python y las librerías |
| Terminal (python kpis.py) | Para el script del reto | Es lo que usará TurisData. |

**Regla de oro del cuaderno:** si algo se comporta raro, Reiniciar entorno y ejecutar todo de arriba abajo.

# Entorno limpio: venv, pip y Git

```
python -m venv .venv          # un entorno aislado por proyecto
source .venv/bin/activate     # (Windows: .venv\Scripts\activate)
pip install pandas scikit-learn # instala librerías solo aquí
git init && git add . && git commit -m "Primer commit"
```

- Un venv por proyecto evita el «en mi ordenador funciona».
- requirements.txt anota las versiones para reproducirlo.
- Un commit = una foto del trabajo con un mensaje que dice qué cambia y por qué.

# Cuaderno S04_01 · Variables, tipos y operadores

```
noches, precio_noche, tasa = 5, 72.50, 1.50
total_estancia = noches * (precio_noche + tasa)   # Ejercicio 1
print(f"{noches} noches: {total_estancia:.2f} EUR")  # f-string
```

- Tipos básicos: int, float, str, bool. Operadores: + - * / // %.
- Comparar con == (no con =, que asigna) y combinar con and, or, not.
- Ejercicio 2: limpiar canal_bruto con strip(), lower() y replace() encadenados.

**Si vienes de Java:** no hay ; ni llaves, los bloques se marcan con sangría de 4 espacios y los tipos los lleva el valor.

# Decidir y repetir: if, for, while

```
if tarifa == "flexible" and deposito == "sin_deposito":
    riesgo = "alto"                  # Ejercicio 3
elif tarifa == "flexible" or deposito == "sin_deposito":
    riesgo = "medio"
else:
    riesgo = "bajo"

total, precio = 0, 60                # Ejercicio 4: 60, 65, 70…
for noche in range(noches_estancia):
    total += precio + 5 * noche
```

- for recorre una secuencia; while repite mientras se cumpla algo (¡que algo cambie dentro!).
- Mini-reto del cuaderno: precio_final con 10 % de descuento a partir de 7 noches y 5 % extra por canal directo.

# Funciones: encapsular un cálculo

```
def clasificar_antelacion(dias):
    """Devuelve 'ultima_hora', 'anticipada' o 'muy_anticipada'."""
    if dias < 30:
        return "ultima_hora"
    return "anticipada" if dias < 90 else "muy_anticipada"

def precio_final(noches, precio_noche, descuento=0.0):
    return round(noches * precio_noche * (1 - descuento), 2)
```

- Cuaderno S04_02: parámetros por defecto, *args (maximo_ingreso(200, 450.5)), lambda y módulos (datetime, statistics).
- Cada función lleva docstring y una prueba assert que la comprueba.

# Ahora, a practicar

Lo que viene a continuación está en tu repositorio.

| | |
|---|---|
| CUADERNO · Entorno, variables, tipos, if, for y while (sprints/S04/notebooks/S04_01_entorno_sintaxis) | 45 min |
| CUADERNO · Funciones, *args, lambda y módulos (sprints/S04/notebooks/S04_02_funciones_modulos) | 45 min |
| CUADERNO · Listas, tuplas, diccionarios y conjuntos (sprints/S04/notebooks/S04_03_estructuras_datos) | 45 min |
| CUADERNO · CSV, JSON y try/except (prepara leer_reservas) (sprints/S04/notebooks/S04_04_ficheros_excepciones) | 35 min |